The Face Mask Detection System is architected to support two distinct delivery pathways tailored to the technical requirements and regulatory constraints of different customer personas:

• Pathway 1 (End-Users & General Public): Zero-install web delivery hosted via Streamlit Community Cloud. Users interact with the AI application through a secure, public HTTPS hyperlink on their browser without downloading any software or managing code.

Streamlit Community Cloud allows free, continuous cloud hosting directly synchronized with a GitHub repository. Follow these exact steps to deploy or re-deploy the application:

Step 2.1: Repository Preparation on GitHub

The project repository must be public and contain the following essential files at the root level:

• src/app.py: The main entry point containing the Streamlit web dashboard interface.

• requirements.txt: Defines all pinned Python dependencies (opencv-python-headless, numpy<2.0.0, Pillow, streamlit, pytest).

• packages.txt: Specifies low-level Linux Debian packages (libgl1, libglib2.0-0) required by OpenCV.

• models/: Contains the Caffe ResNet-10 deep learning weights (deploy.prototxt and caffemodel) and haarcascade fallback XML.

Step 2.2: Launching on Streamlit Cloud

1. Navigate to https://share.streamlit.io and sign in with your GitHub account.

2. Click the 'Create app' or 'New app' button in the upper-right corner.

3. Configure the three deployment parameters:

- Repository: Select your GitHub repository (e.g., pankajatd/faceMask_detection_with_7SDLC_MultiAgents).

- Branch: Set to 'main'.

- Main file path: Enter 'src/app.py'.

4. Click 'Deploy!'. Streamlit Cloud provisions an isolated cloud runner, installs packages from packages.txt and requirements.txt, and starts the web service.

Step 2.3: Managing and Sharing Your Public Link

Docker eliminates the classic 'it works on my machine' issue by bundling the application, exact Python interpreter, low-level OS graphics drivers, and model weights into an immutable shipping container.

Step 3.1: Anatomy of the Dockerfile

Our Dockerfile defines a minimal, hardened container image:

• FROM python:3.10-slim: Initializes a stripped-down, ~150MB Debian Linux environment optimized for speed and security.

• RUN apt-get install libgl1 libglib2.0-0: Installs the C++ graphics shared objects essential for OpenCV matrix processing.

• COPY requirements.txt & RUN pip install: Pre-installs all pinned dependencies in an isolated layer.

• COPY models/ src/ tests/: Embeds the AI neural network weights and application source code.

• EXPOSE 8501: Opens the networking port used by Streamlit.

• ENTRYPOINT: Executes 'streamlit run src/app.py --server.port=8501 --server.address=0.0.0.0'.

Step 3.2: Why docker-compose.yml is Used

The docker-compose.yml file serves as a persistent configuration shortcut. It pre-configures port mapping (8501:8501), container naming (facemask_app), and restart policies, allowing developers and customers to launch the container with a single command.

Step 3.3: How to Build and Run Docker

1. Ensure Docker Desktop is launched on your machine and showing 'Engine running' (green).

2. Open Command Prompt (cmd) or PowerShell in the project root directory:

cd C:\Pankaja\AUTOMATED_SDLC_AGENT_PROJECTS\faceMask_detection

3. Execute the single build-and-run command:

docker-compose up -d --build

4. To stop the running container at any time, execute:

After running 'docker-compose up -d --build', verify container operation through three independent verification methods:

Method A: Terminal Status Check

In Command Prompt or PowerShell, execute:

docker ps

Look for the container named 'facemask_app'. The status should display 'Up X seconds' with port forwarding '0.0.0.0:8501->8501/tcp'.

Method B: Browser Dashboard Test

Open Chrome, Edge, or Firefox and navigate to:

http://localhost:8501

The web dashboard will render. Select a demo image (e.g., 'Two Nurses Gloves') or upload an image to confirm facial localization and green/red bounding boxes appear with real-time percentages.

Method C: Docker Desktop Visual Inspection

Depending on the client category, deliver the appropriate assets according to the following guidelines:

Delivery Model A: Enterprise / Corporate Client (Docker Handover)

• Target Audience: Hospital networks, corporate security departments, bank branches, factory compliance teams.

• Delivery Package: Provide the GitHub repository link (or deliver a secure .zip archive) containing:

- **Dockerfile and docker-compose.yml**
- **models/ (trained ResNet-10 Caffe weights and Haar fallback)**
- **src/ (app.py, detector.py, utils.py)**
- **requirements.txt and README.md**

• Customer Action: The client's IT administrator extracts the archive onto their internal server and executes:

docker-compose up -d --build

The system runs privately on their internal network (e.g., http://10.0.0.50:8501) with zero internet exposure.

Delivery Model B: End-User / Client Evaluation (Public Web Hyperlink)

• Target Audience: Recruiters, non-technical clients, testers, and remote evaluators.

• Delivery Package: Share only the live Streamlit Cloud hyperlink:

https://[your-app-name].streamlit.app

• Public Cloud Deployment (End-Users): share.streamlit.io -> Main File: src/app.py -> Access via .streamlit.app URL

• On-Premise Docker Deployment (Enterprises): docker-compose up -d --build -> Access via http://localhost:8501

• Verification & Status Check: docker ps -> Look for facemask_app with port 8501

• Container Termination: docker-compose down

• Automated Test Suite Execution: pytest -v tests/ (25/25 Passing)